

Solutions last updated: Tuesday, May 20, 2025

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT the Name and Student ID of the person to your left: _____

PRINT the Name and Student ID of the person to your right: _____

PRINT the Name and Student ID of the person in front of you: _____

PRINT the Name and Student ID of the person behind you: _____

You have 170 minutes. There are 10 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	8	9	10	Total
Points:	19	8	13	9	10	12	12	9	8	0	100

For questions with **circular bubbles**, you may select only one choice.

- ☐ Unselected option (Completely unfilled)
- ☒ Don't do this (it will be graded as incorrect)
- ☒ Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ☐ You can select
- ☐ multiple squares
- ☒ (Don't do this)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

If an answer requires hex input, you must only use capitalized letters (0xBOBACAFE instead of 0xb0bacafe). For hex and binary, please include prefixes in your answers unless otherwise specified, and do not truncate any leading 0's. For all other bases, do not add any prefixes or suffixes.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam.

Acknowledge that you have read and agree to the honor code above and sign your name below:

Q1 61Clutter 🗑️

(19 points)

Q1.1 0xCAFEF00D 🥤

For all of Q1.1, assume that numbers are represented with 16 bits.

Q1.1.1 (1 point) Convert 0xCAFE to 16-bit binary.

0b1100 1010 1111 1110

Solution: We directly convert each hex digit to its binary counterpart and concatenate the resulting bits:

0b 1100 1010 1111 1110
 C A F E

Q1.1.2 (1 point) What is 0xCAFE - 0xB0BA in hexadecimal, treating both as unsigned numbers?

0x1A44

Solution: We can perform subtraction in much the same way we would in decimal:

 0xCAFE
- 0xB0BA

 0x1A44

Q1.1.3 (1 point) If 0xCAFE and 0xB0BA are two's complement numbers, is 0xCAFE - 0xB0BA negative when interpreted as a two's complement number?

☐ True ☒ False

Q1.2 (2 points) Which of the following n -bit number representations have a range of $[-2^{n-1} + 1, 2^{n-1} - 1]$ (inclusive)? Select all that apply.

☐ Biased

☐ Two's Complement

☒ Sign Magnitude

☒ Ones' Complement

☐ Unsigned

☐ None of the above

Q1.3 CALLing After Me

Q1.3.1 (1 point) Which stage of CALL generates a relocation table?

☐ Compiler

☒ Assembler

☐ Linker

☐ Loader

Q1.3.2 (1 point) Consider these statements about the assembly stage of CALL. Which are true? Select all that apply.

- ☒ Outputs a file containing the text segment and data segment
- ☐ Converts high level language code to assembly language code
- ☐ Resolves all label and data references
- ☐ Outputs a file containing pseudo-instructions
- ☐ None of the above

Q1.4 *Ca(l)c(he)ulations* 🇺🇸

Q1.4.1 (2 points) Given a 64 KiB 4-way set-associative cache with 8-byte cache lines, how many sets does the cache have? How many cache lines are there in each set? Express each answer as a power of 2.

Sets: 2^{11}

Lines: 2^2

Q1.5 *Cache Browns* 🍪

We want to build a multi-level cache system. We have several options for caches to place at each level to optimize the average memory access time (AMAT).

Suppose that you have the following caches to choose from:

- (A) Direct-mapped cache with a hit rate of 80% and a hit time of 20 ns
- (B) Fully-associative cache with a hit rate of 30% and a hit time of 10 ns
- (C) Direct-mapped cache with a hit rate of 25% and a hit time of 20 ns.

Further, assume that the main memory requires a hit time of 100 ns.

Q1.5.1 (2 points) Which cache organization results in the lowest AMAT?

- ☒ (A) $\rightarrow$ (B) $\rightarrow$ Main Memory ☐ (B) $\rightarrow$ (A) $\rightarrow$ Main Memory

Q1.5.2 (2 points) Now, suppose we have the following three-level cache organization:

(A) $\rightarrow$ (C) $\rightarrow$ (B) $\rightarrow$ Main Memory

What is the AMAT of this new configuration?

36

Solution:

Q1.6 *I Don't Care (I ❤️ It)* 🎵

For the following question, refer to the Single-Cycle RISC-V Datapath on the 61C Reference Card.

Q1.6.1 (2 points) Which of the following control signals can be set to “Don’t Care” (*) for at least one instruction while preserving expected behavior? Select all that apply.

- | | | | |
|---|--|---------------------------------|--|
| ☐ PCSel | ☒ ImmSel | ☐ RegWEn | ☒ BrUn |
| ☐ BSel | ☒ ASel | ☐ ALUSel | ☐ MemRW |
| ☒ WBSel | ☐ None of the above | | |

(Question 1 continued...)

Given the following Valgrind excerpt, answer the following debugging questions.

```
==945103== Memcheck, a memory error detector
==945103== Invalid read of size 1
==945103==    at 0x4007A4: bad_ex3 (valgrind_examples.c:25)
==945103==    by 0x40090D: main (valgrind_examples.c:62)
==945103== Address 0x5222045 is 0 bytes after a block of size 5 alloc'd
==945103==    at 0x4C39185: malloc (vg_replace_malloc.c:442)
==945103==    by 0x40077B: bad_ex3 (valgrind_examples.c:21)
==945103==    by 0x40090D: main (valgrind_examples.c:62)
==945103==
==945103== HEAP SUMMARY:
==945103==    in use at exit: 0 bytes in 0 blocks
==945103==   total heap usage: 2 allocs, 2 frees, 1,029 bytes allocated
==945103== All heap blocks were freed -- no leaks are possible
==945103== For lists of detected and suppressed errors, rerun with: -s
==945103== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Q1.7 (1 point) What line number of `valgrind_examples.c` contains the invalid read? Your answer should be a positive integer with no units.

25

Q1.8 (1 point) The output “Address 0x5222045 is 0 bytes after a block of size 5 alloc’d” tells us that:

- ☒ A block of size 5 was successfully allocated, but address 0x5222045 is outside that block.
- ☐ Address 0x5222045 was freed after a block of size 5 was allocated.
- ☐ 5 bytes were intended to be allocated at address 0x5222045 but only 0 bytes were actually allocated.
- ☐ None of the above

Q1.9 (2 points) Consider a program where 90% of the program can be parallelized. If we were to have infinitely many resources dedicated to running this program, what is the maximum possible speedup?

10x Speedup

Q2 61Cookies 🍪

(8 points)

Sophie is baking RISC-V cookies. Her oven has a maximum capacity, which we will call *batch size*, and it takes 13 minutes to bake a single batch of cookies. For example, if Sophie is baking 10 cookies with a batch size of 5, it will take her 26 minutes for 2 batches. If Sophie is baking 11 cookies with a batch size of 5, it will take her 39 minutes for 3 batches (the last batch will be one cookie).

Given the total number of cookies in register **a0** and the batch size in register **a1**, write the recursive RISC-V function **bake_cookies** to calculate the minimum time (in minutes) it takes to bake all the cookies.

(8 points) Implement **bake_cookies**.

bake_cookies : Returns the minimum time in minutes it takes to bake all a0 cookies with a batch size of a1 .		
Arguments	a0	The number of cookies to bake
	a1	Batch size
Return value	a0	The minimum time in minutes to bake all cookies

You may assume that **R[a0]** and **R[a1]** represent signed integers with value greater than 0.

```

1  bake_cookies:
2      bge x0 a0 base_case
           Q2.1
3      addi sp sp -4
           Q2.2
4      sw ra 0(sp)
           Q2.3
5      sub a0 a0 a1
           Q2.4
6
7      jal bake_cookies
8
9      addi a0 a0 13
           Q2.5
10     lw ra 0(sp)
           Q2.6
11     addi sp sp 4
           Q2.7
12     jr ra
13
14  base_case:
15     li a0 0
           Q2.8
16     jr ra

```

(Question 2 continued...)

Solution: Alternate solutions were possible (because `a0` and `a1` are both guaranteed to be > 0): `bge a1 a0 base_case` followed by `li a0 13` at the end.

Q3 Balatro 🎴

(13 points)

Andrew, Alonzo, Alex, and Marie-Anne love the card game Balatro so much that they want to recreate a small version of it! Balatro is a card game that calculates your final score by multiplying your **chips** (the sum of every `cards[i].value`) by your **mult** (score multiplier). `joker_t` functions are applied to the current game and modify **chips** and **mult**.

```
1 #define NUM_CARDS 5
2 #define NUM_JOKERS 5
3
4 typedef struct card {
5     uint32_t suit;    // 0 = HEARTS, 1 = DIAMONDS, 2 = CLUBS, 3 = SPADES
6     uint32_t value;
7 } card_t;
8
9 // joker_t is the type of a function that takes in card_t*, float*, and float*
10 // as arguments, and has no returns.
11 // A valid joker_t function takes in HAND, MULT, and CHIPS, and
12 // updates MULT and CHIPS in place by looking at the cards in HAND.
13 typedef void(*joker_t)(card_t *, float *, float *);
14
15 typedef struct game {
16     float mult;
17     float chips;
18     card_t cards[NUM_CARDS];
19     joker_t joker_fns[NUM_JOKERS];
20 } game_t;
```

(4 points) Implement `same_suit_joker` according to the specification below:

same_suit_joker: Increments mult by 10 in-place if all cards in hand have the same suit.		
Arguments	card_t *hand	Array of cards in hand
	float *mult	Pointer to mult of a game_t struct
	float *chip	Pointer to chips of a game_t struct
Return value	void	

```
1 void same_suit_joker(card_t *hand, float *mult, float *chips) {
2     int suit = hand[0].suit;
3     for (int i = 1; i < NUM_CARDS; i++) {
4         if (hand[i].suit != suit) {
5             return;
6         }
7     }
8     *mult += 10;
9 }
```

Q3.1
Q3.2
Q3.3

Solution: Blank 3.1 gets the `suit` of the first card in the hand, and then the `if` statement on line 4 leads to an early return if any of the subsequent suits differ from that of the first card. Finally, if all the cards are the same, we must dereference `mult` in order to modify it in-place.

Grading:

- Q3.1
 - +1 point for a correct expression
 - +0.5 points for off-by-one dereference errors (e.g. applying `->` to a non-pointer), or for operator precedence errors (e.g. `*hand.suit`).
 - -0.1 points for unambiguous naming errors, such as using `cards` instead of `hand`
 - No points were awarded for explicitly applying the dereference operator to any expression that evaluated to a non-pointer (e.g. `*hand[0].suit`)
- Q3.2
 - +2 points for a correct expression
 - +1 point for off-by-one dereference errors (e.g. applying `->` to a non-pointer), or for operator precedence errors (e.g. `*(hand + i).suit`).
 - -0.1 points for unambiguous naming errors, such as using `cards` instead of `hand`
 - No points were awarded for explicitly applying the dereference operator to any expression that evaluated to a non-pointer
- Q3.3
 - All or nothing

(7 points) Now, implement `compute_score` according to the specification below:

compute_score: Computes the score a <code>game_t</code> struct would obtain		
Arguments	<code>game_t *game</code>	A pointer to a <code>game_t</code> struct
Return value	<code>float</code>	The score obtained by the current <code>game_t</code> state.

```

1 float compute_score(game_t *game) {
2     // Add the value of each card to game->chips
3     for (int i = 0; i < NUM_CARDS; i++) {
4
5         game->chips += game->cards[i].value;           Q3.4
6     }
7     // Apply each joker_t function to this game
8     for (int j = 0; j < NUM_JOKERS; j++) {
9
10        game->joker_fns[j](                             Q3.5
11            game->cards, &game->mult, &game->chips      Q3.6      Q3.7      Q3.8
12        );
13    }
14    return game->chips * game->mult;
15 }
```

(Question 3 continued...)

In the below code, `populate_cards` fills in the `CARDS` array in the `GAME_T` struct with random cards. Andrew wants to use threads to speed up this process and has provided their implementation below:

```
1 // Generates a random card
2 card_t random_card() { /* Implementation omitted */ }
3
4 // Randomly fills the CARDS array in the GAME_T struct.
5 void populate_cards(game_t *game) {
6     int left = 0;
7     int right = NUM_CARDS - 1;
8
9     omp_set_num_threads(2);
10
11     #pragma omp parallel
12     {
13         int tid = omp_get_thread_id();
14         while (left <= right) {
15             if (tid == 0) {
16                 game->cards[left] = random_card();
17                 left++;
18             } else {
19                 game->cards[right] = random_card();
20                 right--;
21             }
22         }
23     }
24 }
25
26 game_t *initialize_game() {
27     game_t *game = malloc(sizeof(game_t));
28     /* ... */
29     populate_cards(game);
30     /* ... */
31     return game;
32 }
```

Q3.9 (1 point) What are the minimum and maximum possible values for `left` when this function returns?

Min: 0

Max: NUM_CARDS

Q3.10 (1 point) In the above code, `populate_cards` is missing a call to a memory allocation function to initialize the `cards` array in `game`.

☐ True ☒ False

Q4 Iowa [S]outh [N]orth [W]est 🌱👉👈

(9 points)

For this problem, use the single-cycle datapath on the 61C Reference Card. You want to support the following new instruction:

`lowasnw rd rs2 imm(rs1)` (load old word and store new word)

$R[rd] = M[R[rs1] + imm]$

$M[R[rs1] + imm] = R[rs2]$

Q4.1 (3 points) What changes would you need to make to our single-cycle datapath to implement the `lowasnw` instruction with as few changes as possible? Select all that apply.

- ☒ Create a new instruction type and update the ImmGen
- ☐ Add a new output to the RegFile for a third value
- ☐ Add another WriteData and WriteIndex input to the RegFile
- ☐ Add a third input into ALU and update any relevant selector/control logic
- ☐ Allow the ALU to send out more than 1 output and update any relevant selector/control logic
- ☐ Allow the DMEM to be able to write then read at the same clock cycle and update any relevant selector/control logic
- ☒ Allow the DMEM to be able to read then write at the same clock cycle and update any relevant selector/control logic
- ☐ Add a new input to the WBMux and update any relevant selector/control logic
- ☐ None of the above

For each of the following control signals, indicate the value it should have for `lowasnw`.

Q4.2 (1 point) PCSel

- ☒ PC + 4
- ☐ ALUOut
- ☐ Doesn't matter

Q4.3 (1 point) RegWEn

- ☐ Write disabled
- ☒ Write enabled
- ☐ Doesn't matter

Q4.4 (1 point) ASel

- ☒ rs1
- ☐ PC
- ☐ Doesn't matter

Q4.5 (1 point) BSel

- ☐ rs2
- ☒ imm
- ☐ Doesn't matter

Q4.6 (1 point) BrUn

- ☐ Signed
- ☐ Unsigned
- ☒ Doesn't matter

Q4.7 (1 point) WBSel

- ☐ PC + 4
- ☐ ALUOut
- ☒ MemReadData
- ☐ Doesn't matter

(10 points)

The diagram illustrates the internal architecture of a 5-stage MIPS processor. The stages are labeled at the top: IF (Instruction Fetch), ID (Instruction Decode), BC (Branch Compare), ALU (Arithmetic Logic Unit), MEM (Memory Access), and WB (Write Back). Below the stage labels, the internal components and data flow are shown:

- IF Stage:** The instruction (*inst*) is fetched from memory. The PC (Program Counter) is updated to *PC+4*. The instruction is split into *addr* (address) and *inst* (instruction).
- ID Stage:** The instruction is decoded. The register file (*Reg[]*) is accessed to read *rd* (destination register) and *rs1*, *rs2* (source registers). The instruction is split into *inst* (instruction) and *imm* (immediate).
- BC Stage:** The branch comparator (*Branch Comp*) compares the values in *rs1* and *rs2* against the branch condition (*BrUn*, *BrEq*, *BrLt*, *BrSlt*). The result is used to update the PC to *PC+4* or the branch target address.
- ALU Stage:** The ALU performs the operation specified by the instruction (*ALUOp*) on the values in *rs1* and *rs2*. The result is used to update the PC to *PC+4* or the branch target address.
- MEM Stage:** The data memory (*DMEM*) is accessed. The address (*addr*) is used to read data (*rdata*) or write data (*wdata*). The instruction is split into *inst* (instruction) and *imm* (immediate).
- WB Stage:** The result of the ALU operation is written back to the register file (*Reg[]*). The instruction is split into *inst* (instruction) and *imm* (immediate).

The diagram also shows the internal components of the ALU, including the ALU Mux, ALU, and ALU Out. The ALU Mux selects between the values in *rs1* and *rs2* based on the ALU operation. The ALU performs the operation and outputs the result to the ALU Out. The ALU Out is used to update the PC to *PC+4* or the branch target address.

```

1  addi t0 x0 5
2  addi t1 x0 5
3  add a5 a5 a5
4  add a6 a6 a6
5  add a7 a7 a7
6  beq t0 t1 label
7  add t3 t0 t0
8  add t4 t0 t0
9  label:
10         addi t1 t0 t1
11         addi t2 t1 t0

```

(Question 5 continued...)

Assuming all hazards are resolved **only** through stalling, identify the **first three hazards in program order** that occur when the given program is run on the 6-stage pipeline. Assume you **cannot** write-then-read from the same register in the same clock cycle.

For each hazard, indicate the type of hazard and the number of stalls needed to fix the hazard. Assuming there are fewer than three hazards, select “None”. If no stalls are needed, answer “0”.

Q5.1 (2 points) Hazard 1:

- ☐ Structural ☐ Control
☒ Data ☐ None

Stall(s): 1

Q5.2 (2 points) Hazard 2:

- ☐ Structural ☒ Control
☐ Data ☐ None

Stall(s): 4

Q5.3 (2 points) Hazard 3:

- ☐ Structural ☐ Control
☒ Data ☐ None

Stall(s): 4

For the remainder of this question, the forwarding paths below have been added to the datapath.

- WB → ALU
- WB → BC

Consider the following RISC-V code:

```
1 lw a3 0(sp)
2 addi t0 a3 5
3 beq t1 t0 Label
```

For each hazard below, indicate the number of stalls needed to resolve the hazard.

Q5.4 (2 points) The hazard between lines 1 and 2

Stall(s): 1

Q5.5 (2 points) The hazard between lines 2 and 3

Stall(s): 2

Q6 EVX

(12 points)

Noah misses cheap eggs! He wants to use vector instructions to analyze egg prices. To do this, he uses the special EVX (Exam Vector eggXtension) to look through prices and zero out all prices greater than or equal to a certain limit.

filter: Sets all elements of prices greater than or equal to limit to zero.		
Arguments	int32_t limit	The minimum price that should be filtered out of prices
	int32_t *prices	An array of prices
	size_t len	The number of elements in prices
Return value	void	

The following C code is the non-vectorized implementation:

```

1 void filter(int32_t limit, int32_t *prices, size_t len) {
2     for (int i = 0; i < len; i++) {
3         if (prices[i] >= limit) {
4             prices[i] = 0;
5         }
6     }
7 }
```

You have access to the following SIMD operations. A single vector is a **256-bit** vector register capable of holding **eight** 32-bit integers.

- **vector vec_load(uint32_t *A):** Loads eight integers at memory address **A** into a vector.
- **void vec_store(uint32_t *dst, vector src):** Stores **src** to **dst**.
- **vector vec_setnum(int32_t num):** Creates a vector where every element is equal to **num**.
- **vector vec_and(vector A, vector B):** Returns the result of ANDing **A** and **B** element-wise.
- **vector vec_add(vector A, vector B):** Returns the result of adding **A** and **B** element-wise.
- **vector vec_mul(vector A, vector B):** Returns the result of multiplying **A** and **B** element-wise.
- **vector vec_srai(vector A, int count):** Returns the result of right-shifting (sign-extended) each element in **A** by the number of bits specified in **count**.

(Question 6 continued...)

(12 points) Using the provided SIMD operations, complete a vectorized implementation of `filter` that provides the greatest performance increase. For full credit, each line of code can have at most 2 SIMD instructions. (e.g. you may not use any SIMD instruction on line 8)

```
1 void filter(int32_t limit, int32_t *prices, size_t len) {
2     vector vec_neg = vec_setnum(-1);
3     vector vec_limit = vec_setnum(limit);
4     for (int i = 0; i < len / 8 * 8; i += 8) {
5         vector vec_arr = vec_load((uint32_t *) (prices + i));
6         vector vec_mask = vec_add(vec_arr, vec_mul(vec_neg, vec_limit));
7         vec_mask = vec_srai(vec_mask, 31);
8         vec_store((uint32_t *) (prices + i), vec_and(vec_mask, vec_arr));
9     }
10    for (int i = len / 8 * 8; i < len; i += 1) {
11        if (prices[i] >= limit) {
12            prices[i] = 0;
13        }
14    }
15 }
```

Q7 Page Table Talk

(12 points)

For Q7.1 — Q7.2, consider a system with 4 GiB of virtual memory, 1 GiB of physical memory, and a 1 KiB page size.

Q7.1 (3 points) How many bits are in the Virtual Page Number (VPN), Physical Page Number (PPN), and Page Offset?

VPN: 22 bit(s)

PPN: 20 bit(s)

Offset: 10 bit(s)

Q7.2 (2 points) Assume each page table entry (PTE) has 12 bits of metadata. What is the size of a page table? Express your answer as a power of 2.

Page Table Size: 2^{24} bytes

The rest of this page left intentionally (mostly) blank.

(Question 7 continued...)

For the remaining questions, assume that the Page Table Entries (PTEs) for each process are stored at consecutive increasing addresses in memory. That is, the address of the PTE corresponding to a given Virtual Page Number (VPN) can be computed by:

$$\text{Page Table Base Address} + \text{VPN} \times (\text{size of PTE in bytes})$$

Q7.3 (2 points) Consider a system with 6 VPN bits, 14 PPN bits, 10 offset bits, and **16-byte PTEs**. If the Page Table Base Address for this process begins at the physical address **0x1000**, at what physical address will we find the page table entry for the virtual address **0x5999**?

0x1160

Finally, consider a new system where each PTE is 32 bits and is formatted as follows:

1 Valid Bit	23 Status Bits	8 PPN Bits
-------------	----------------	------------

(5 points) Complete the RISC-V function **pt_extract** which loads the Page Table Entry (PTE) from the current Page Table and then returns the valid and PPN bits (even if the PTE is invalid). Assume that all memory operations within **pt_extract** operate on physical addresses.

pt_extract: Loads the PTE for the VPN in a1 and returns the Valid and PPN bits.		
Arguments	a0	Page Table Base Address
	a1	VPN
Return value	a0	PPN bits extracted from the PTE
	a1	Valid bit extracted from the PTE

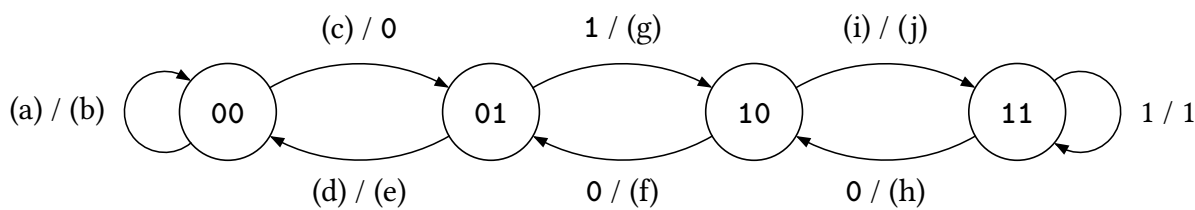
```
1 pt_extract:
2   # load PTE from Page Table with base address in a0
3   slli a1, a1, 2
4   add a0, a0, a1
5   lw a0, 0(a0) # a0 now contains the 32-bit PTE
6   srli a1, a0, 31
7   andi a0, a0, 0xFF
8   jr ra
```

Q8 The Branch Not Taken 🌲

(9 points)

Jamie and Abhi want extra credit on Project 3, so they decide to implement a branch predictor. To do this, they build an FSM (which will be used as a branch predictor later) that follows the specification below:

Branch Predictor FSM		
States	00 (0), 01 (1), 10 (2), 11 (3)	
Output	The MSB of the current state (pre-transition).	
Transitions	If Input == 0:	Decrements state by 1 if possible. Otherwise, remains in the same state.
	If Input == 1:	Increments state by 1 if possible. Otherwise, remains in the same state.



Q8.1 FSM Diagram

Fill out the blanks of the FSM diagram to match the described behavior.

Q8.1.1 (0.5 points) Blank (a)

☒ 0 ☐ 1

Q8.1.2 (0.5 points) Blank (b)

☒ 0 ☐ 1

Q8.1.3 (0.5 points) Blank (c)

☐ 0 ☒ 1

Q8.1.4 (0.5 points) Blank (d)

☒ 0 ☐ 1

Q8.1.5 (0.5 points) Blank (e)

☒ 0 ☐ 1

Q8.1.6 (0.5 points) Blank (f)

☐ 0 ☒ 1

Q8.1.7 (0.5 points) Blank (g)

☒ 0 ☐ 1

Q8.1.8 (0.5 points) Blank (h)

☐ 0 ☒ 1

Q8.1.9 (0.5 points) Blank (i)

☐ 0 ☒ 1

Q8.1.10 (0.5 points) Blank (j)

☐ 0 ☒ 1

Now, they wire up their single-cycle CPU to use the FSM as a branch predictor.

- When the CPU runs a B-type instruction, two things happen:
 - First, `PCSel` is set to the MSB of the FSM's current state.
 - Then, the *actual* branch outcome (Taken = 1, Not Taken = 0) is inputted into the FSM, and the FSM's state is updated accordingly.
- On all other instructions, the FSM is unused and unchanged.

Example:

- Suppose the FSM is in state 01.
- Now, the CPU runs `beq x0 x0 label`.
- `PCSel` is set to 0 (the MSB of the current state), meaning that the FSM predicted "Branch Not Taken"
- However, the actual branch is taken, since `R[x0] == R[x0]`. This sends the input 1 ("Branch Taken") to the FSM, and causes our FSM to transition to state 10.

Q8.2 Mischievous Mispredicting 🐱

Abhi's feeling mischievous; he wants to write code to ensure branch prediction fails as often as possible. In other words, he wants to write code that will result in an infinite series of branch operations that are all predicted incorrectly. Here is Abhi's code so far:

```
1 ### YOUR CODE HERE ###
2 bne x0 x0 0
3 beq x0 x0 -4
```

For each starting state below, fill in Line 1 (### YOUR CODE HERE ###) such that when the above code is run, there is an infinite sequence of mispredicted branch operations. Choose from the following:

1. Provide a single RISC-V instruction (no pseudo-instructions).
2. If an instruction is not necessary to produce an infinite sequence, bubble "Blank — no additional instruction needed".
3. If you believe an infinite sequence of mispredictions is not possible, bubble "Not Possible".

Q8.2.1 (0 points—completed for you) FSM starts at State 00

Hint: We have provided a possible solution for you to help you with later subparts. Trace through the values of `PCSel` and compare that against the FSM's state transition rules to see how the branch predictor outputs an infinite sequence of mispredictions.

`beq x0 x0 8`

- ☐ Blank — no additional instruction needed
☐ Not Possible

Q8.2.2 (2 points) FSM starts at State 01

`beq x0 x0 4`

- ☐ Blank — no additional instruction needed
☐ Not Possible

Q8.2.3 (2 points) FSM starts at State 10

- ☒ Blank — no additional instruction needed
☐ Not Possible

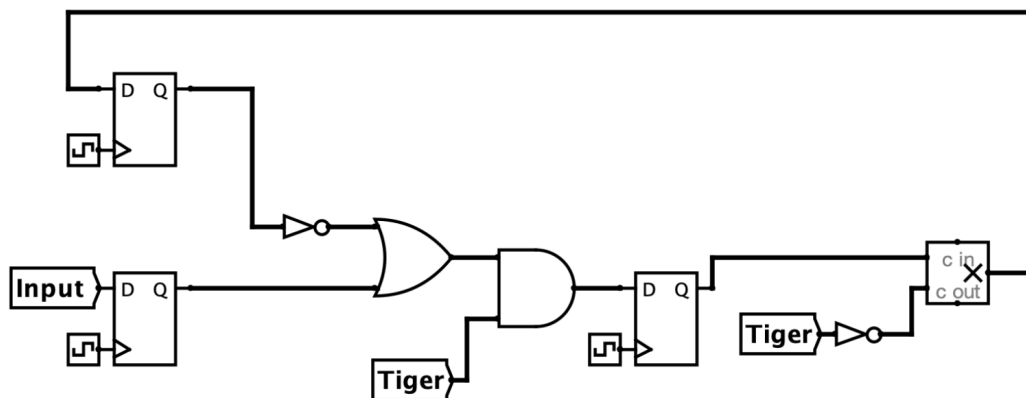
Solution:

Note that alternate solutions were possible for this question as long as they respect the FSM's transition rules: some creative examples use JAL, or check a different branching condition to the provided solution.

Q9 Tiger 🐯

(8 points)

Hoshi, a CS 61C Student, has built the following circuit diagram. **Tiger** is a single-bit control signal that **updates 2 ps after the rising edge of every clock cycle** and remains stable for the rest of the clock cycle. Additionally, **Input will not cause any hold time violations** and is a single-bit value.



$t_{\text{NOT}} = 5 \text{ ps}$, $t_{\text{MUL}} = 100 \text{ ps}$, $t_{\text{OR}} = 50 \text{ ps}$, $t_{\text{clk-to-q}} = 25 \text{ ps}$, $t_{\text{setup}} = 10 \text{ ps}$, $t_{\text{AND}} = ??$

Q9.1 (3 points) If the minimum clock period of this circuit is 136 ps, what is t_{AND} ?

46ps

For the remainder of this question, assume $t_{\text{AND}} = 35 \text{ ps}$ (all other delays remain the same).

Q9.2 (2 points) What is the maximum hold time the registers can have so that there are no hold time violations in the circuit above? Reminder: **Input** will not cause any hold time violations.

37 ps

Another student, Woozi, noticed that the circuit Hoshi built is inefficient and wants to maximally increase the circuit's frequency without changing the behavior of the circuit. Woozi cannot change the delays of any component.

Q9.3 (1.5 points) **Briefly** describe a change Woozi should make. We will only grade the first 10 words of your answer.

Replace multiplier with AND block

Q9.4 (1.5 points) What is the new minimum clock period after implementing your change in Q9.3?

125 ps

This page left intentionally (mostly) blank

The exam continues on the next page.

Q10 Eggcellence

(0 points)

These questions will not be assigned credit; feel free to leave them blank.

Q10.1 How many lifetime OH tickets has Abhi, the 61C TA taken?

Q10.2 Solve this eggciting 61Connections puzzle about 61C Staff!

HEAD TAS

ANTO, ANDREA, MYRAH, ANDREW

NAMES THAT START WITH 'A'

ABHI, ALEX, ARIANA, ALONZO

DISCUSSION TAS

JULIA, CHLOE, NOAH, SASHA

HOMEWORK PARTY STAFF

MARIE-ANNE, ELVA, JAMIE, WEN

Q10.3 If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.